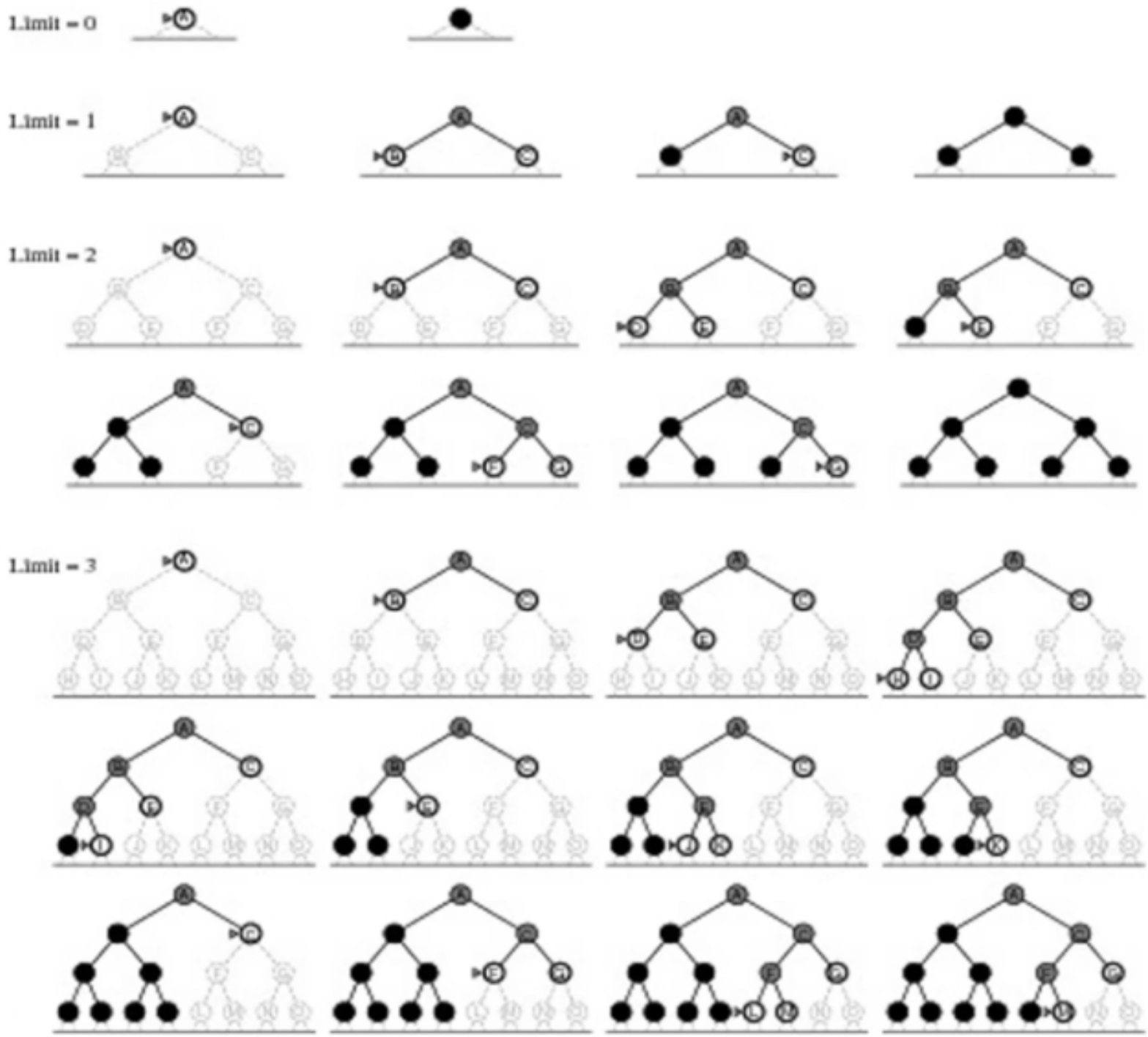

1.limit = 0
1.limit = 1
1.limit = 2
1.limit = 3

Bidirectional Search

- A **forward and backward** breadth first search simultaneously
- b^(d/2) + b^(d/2) is much less than b^d
- **O(b^d/2)** time and space complexity
- A solution will be present at the intersection of frontiers (additional checks needed for optimality)
- How to search backwards ?
  - Need to compute predecessors
- If multiple goals, dummy goal state whose immediate predecessors are the goal states
- Search difficult if goal state is abstract ex: no queen attacks another queen

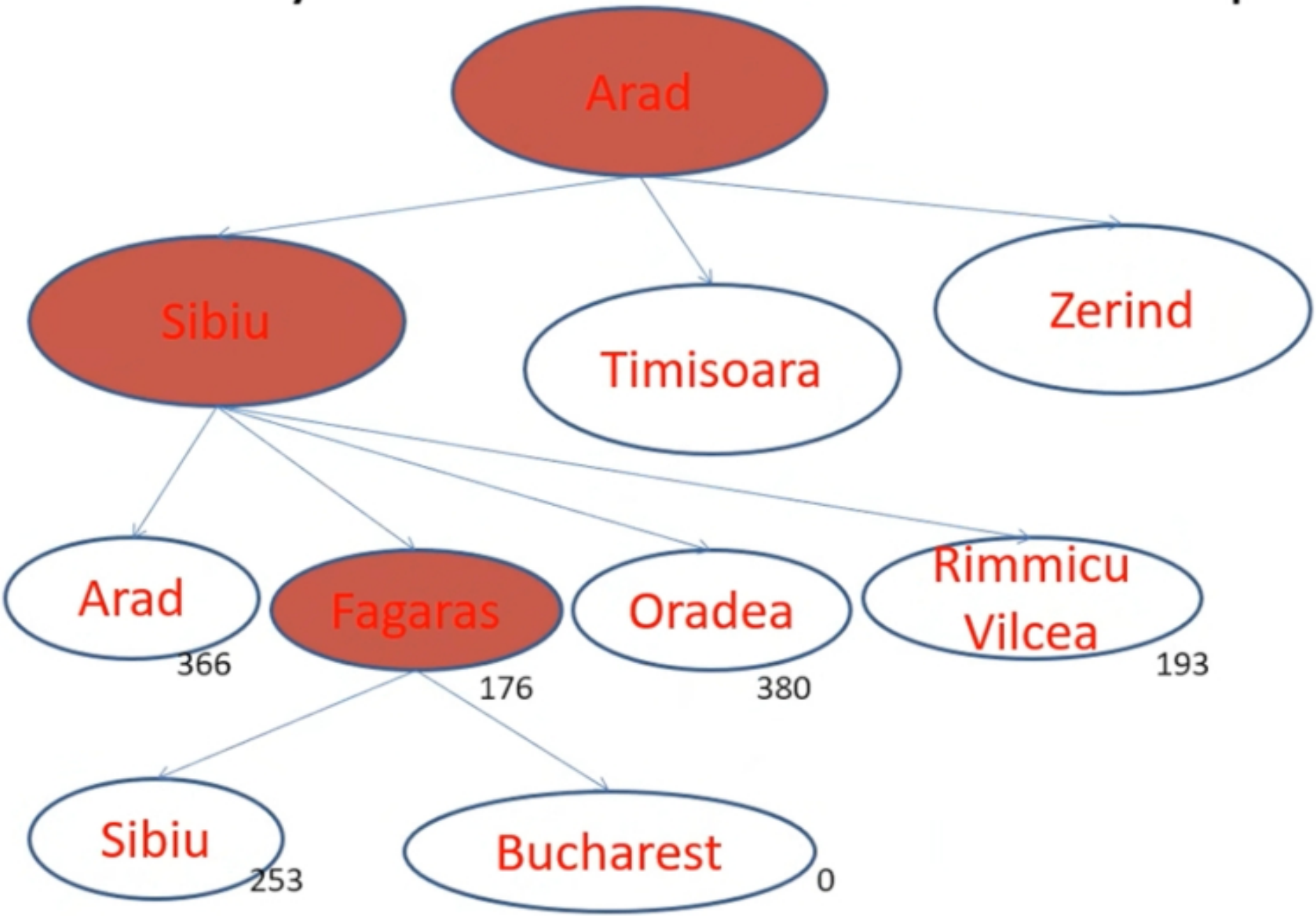
Best First Search
(Chapter 3, Section 5)

- Node selected for expansion based on evaluation function **f(n)**
- Node with lowest cost evaluation expanded first
- Heuristic function **h(n)** as a component of f(n)
- h(n) = *estimated cost of the cheapest path from the state at node n to a goal state*
- **Greedy Best First Search**
- **A* Search**
- **Memory bounded heuristic search**
  - **IDA*, RBFS and SMA***

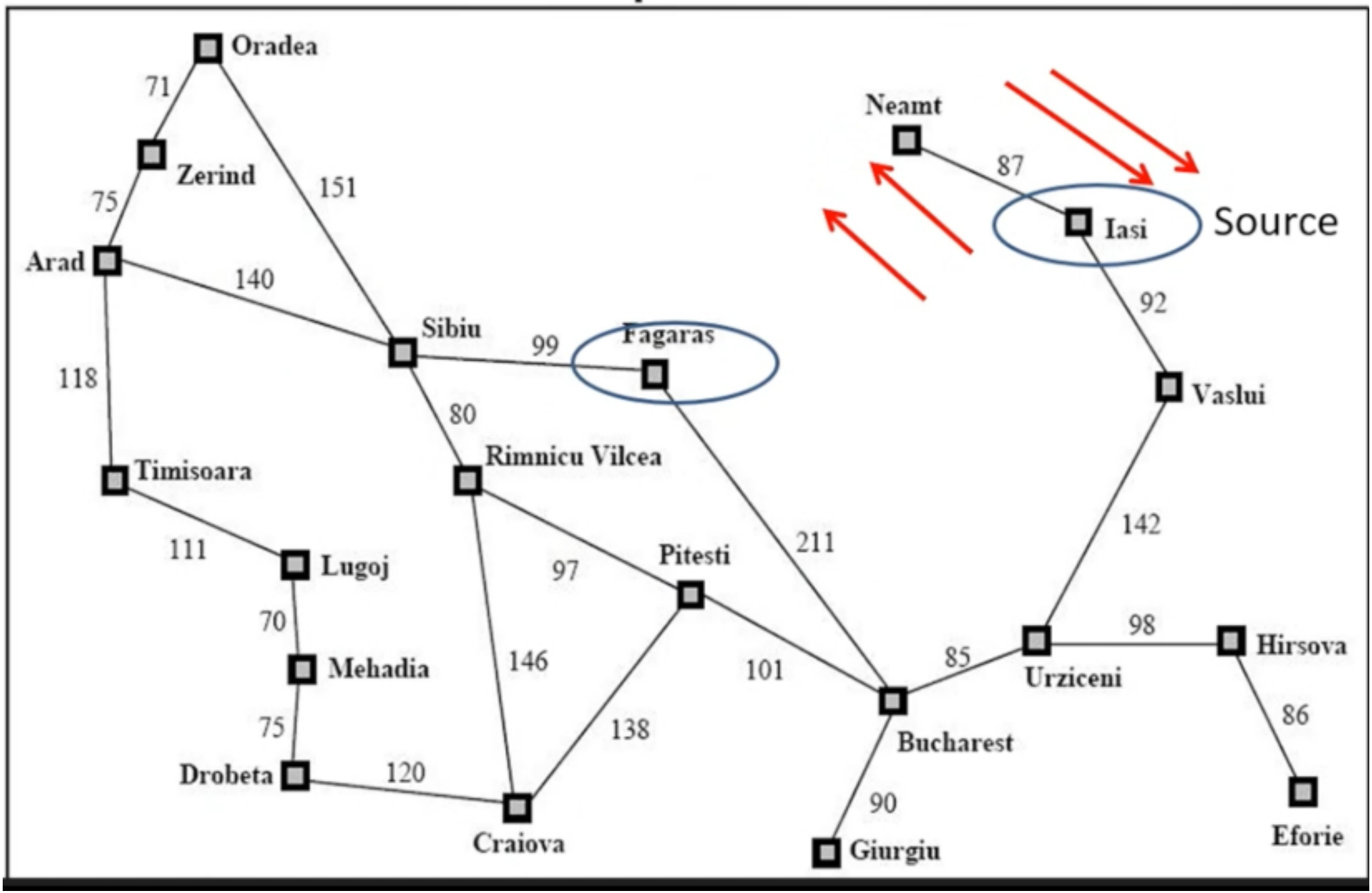
Greedy Best First Search

- Expand node closest to goal
- **f(n) = h(n)**
- For route finding problem straight line distance **h_SLD** can be a heuristic
- h_SLD from Arad to Bucharest is 366 (Next slide)
- **Need not** be optimal
- Tree search is **incomplete** like depth first (infinite loop), graph search is **complete** for finite spaces

Greedy Best First Tree Search Example



Incompleteness



A* Search

- **f(n) = g(n) + h(n)** i.e. *Cost to reach the node + Cost from node to goal*
- f(n) = *Estimated cost of the cheapest solution through n*
- Expand node with **lowest** f(n)
- If h(n) satisfies certain constraints A* search is complete and optimal
- Algorithm identical to **uniform cost search**

A* search example

Open List:

Rimricu Vicea

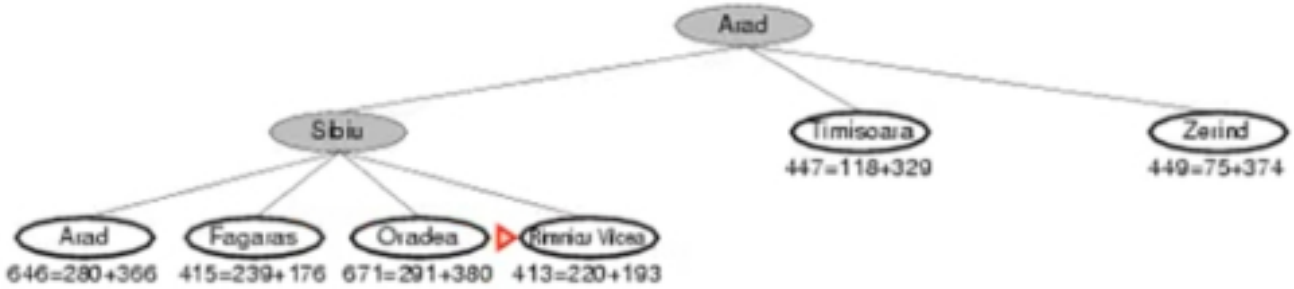
Fagaras

Timisoara

Zerind

~~Arad~~

Oradea



We've been to Arad before. Don't list it again on the open list.

A* search example

Open List:

Bucharest

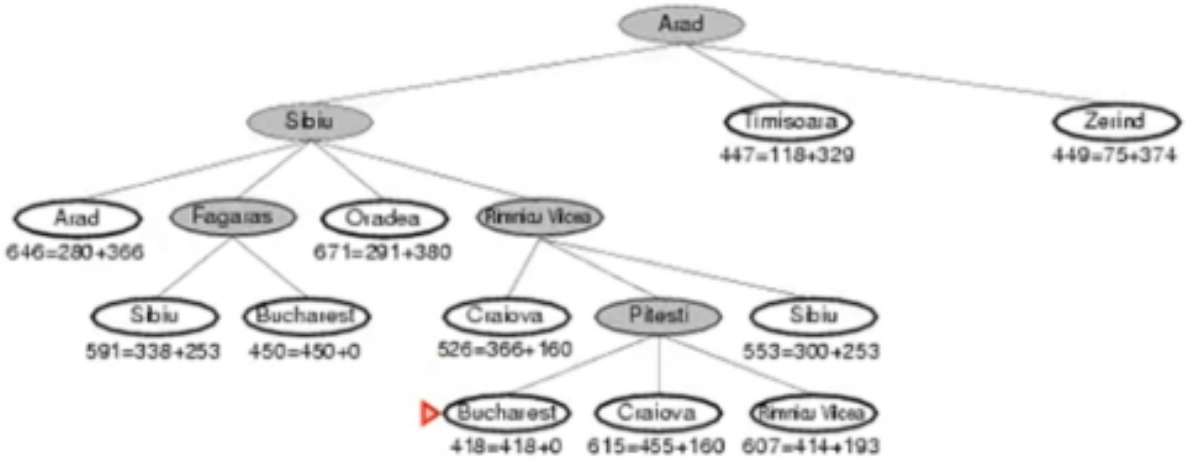
Timisoara

Zerind

Craiova

~~Rimricu Vicea~~

Oradea



We just found a better value for Bucharest; so, it got moved higher in the list. We also found a worse value for Craiova – we just ignore this.

And of course, we ran into Rimricu Vicea again. Since it's already been expanded once, we don't re-add it to the Open List.

Conditions for optimality: Admissibility and Consistency

- **Admissibility**: h(n) never overestimates the cost to reach a goal
- Optimistic in nature since h(n) thinks cost of solving problem is lower than it actually is
- Straight line distance h_LSD is admissible
  - Cannot have something shorter than straight line
- **Consistency (or monotonicity)**: Required for graph search
- h(n) <= c(n,a,n') + h(n')
- Form of triangle inequality – Triangle formed by n, n' and goal G_n
- If there is a route from n to G_n via n' cheaper than h(n) it would violate h(n) as lower bound to reach G_n

Optimality of A*

- **Every** consistent heuristic is admissible
- Consistency is a **stricter** requirement although most heuristics are both admissible and consistent
- *Is h_SLD a consistent heuristic ?*
- Tree search version optimal if h(n) is admissible and graph search optimal if h(n) is consistent
- If A* selects a path p, then p is the shortest or lowest cost path

Proof of optimality for A*

- **Step 1:** If h(n) is consistent, then values of f(n) along any path are non decreasing
- **Step 2:** Whenever A* selects a node n for expansion, the optimal path to that node has been found
- **For Step 1**: For definition of consistency, if n' is successor of n
  - g(n') = g(n) + c(n,a,n')
  - f(n') = g(n') + h(n') = g(n) + c(n,a,n') + h(n') >= g(n) + h(n) = f(n) → f(n') >= f(n)

Proof of optimality for A*

- **For Step 2**: If optimal path to node n was not found before expanding
  - Another frontier node n' would be on the optimal path from start node to n
  - By the graph separation property (frontier separates the explored region of the state space from the unexplored region), since f is non-decreasing n' will have lower cost than n and hence would be selected first
- From steps 1 + 2, sequence of nodes expanded is in **non decreasing** order of f(n)
- Hence the **first goal** node selected must be optimal and later goal nodes will be at least as expensive

A* properties

- A* expands all nodes with $f(n) < C^*$
- A* might expand nodes on goal contour ($f(n) = C^*$) before selecting a goal node
- A* does not expand nodes with $f(n) > C^*$
- A* is complete i.e. will always find a solution
- A* is optimally efficient for any given consistent heuristic
  - No other optimal algorithm is guaranteed to expand fewer nodes than A* given the heuristic
  - If an algorithm does not expand all nodes with $f(n) < C^*$, it may miss the optimal solution
- Hence A* is **complete, optimal and optimally efficient**

A* properties

- Complexity of A* dependent on quality of heuristic function chosen, varies with assumptions made on state space
- **Absolute error Δ** ≡ h*- h (h* is actual cost of getting from root to goal, h the estimated cost)
- **Relative error Є** ≡ (h*-h)/h*
- For tree with reversible actions and single goal, O(b^Δ)
  - With constant step costs, O(b^ Єd) = O((b^ Є)^d)
- With multiple goal states
  - Extra cost proportional to number of goals (whose cost is within factor Є of optimal cost)
- For general graph, exponentially many states with f(n) < C*
- **While computationally expensive to find optimal solutions, usually runs out of space before time**